

Para que este proyecto funcione y puedas presentarlo para el **Trabajo 2**, tienes dos caminos: ejecutarlo de forma **local** (más rápido para probar) o desplegarlo en la **nube de AWS** (que es el objetivo real del trabajo).

Aquí tienes los pasos exactos para ambos escenarios:

Opción 1: Ejecución Local (Simulación de Hadoop)

Esta opción utiliza el archivo

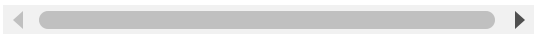
`run_mapreduce.py` para simular cómo funciona un clúster sin necesidad de subir nada a AWS todavía.

1. Preparar el entorno:

Asegúrate de tener instaladas las librerías necesarias:

Bash

```
pip install -r requirements.txt
```



2. Ejecutar el pipeline de datos:

Asegúrate de que el archivo del censo
(`2026_1_Nconductores_...txt`) esté
en la misma carpeta. Ejecuta:

Bash

```
python run_mapreduce.py
```

*Esto leerá el TXT, pasará los datos
por `mapper.py` , los ordenará y
`reducer.py` generará el archivo
`resultados_mapreduce.json` .*

3. Lanzar el Dashboard:

Una vez generado el `.json` , lanza la interfaz visual:

Bash

```
streamlit run dashboard.py
```

Opción 2: Despliegue en AWS (Para el Trabajo de Clase)

Dado que tu `lambda_function.py` está configurado para reaccionar a eventos de S3, el flujo profesional sería:

1. Configurar el almacenamiento (S3)

- Crea un **Bucket en S3** (ej: `censo-conductores-86`).
- Crea dos carpetas dentro: `/input` y `/output` .
- Sube `mapper.py` y `reducer.py` a la raíz del bucket (la Lambda los buscará allí según tu código).

2. Crear la Función Lambda

- Crea una nueva función Lambda en Python 3.12.
- Copia el código de tu archivo `lambda_function.py` .

- **Configuración Crítica:** Aumenta el *Timeout* a 5 minutos y la *Memoria* a 1024 MB (el censo es pesado para los 128 MB por defecto).
- Dale permisos a la Lambda (IAM Role) para leer y escribir en S3.

3. Configurar el Disparador (Trigger)

- En la consola de la Lambda, añade un **S3 Trigger**.
- Configúralo para que se active solo cuando se suba un archivo a la carpeta `input/`.

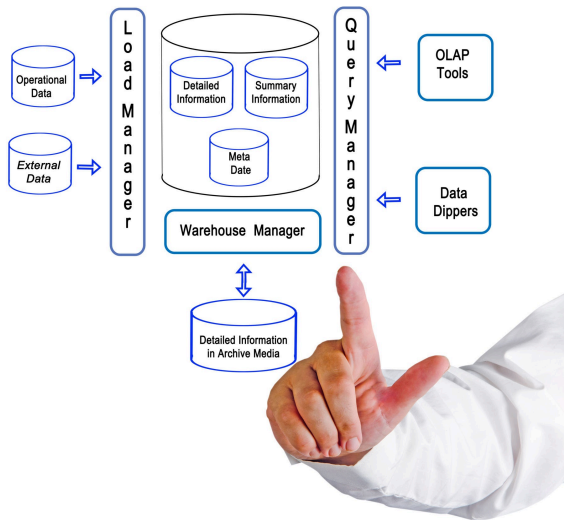
4. Ejecución

- Sube el archivo `2026_1_Nconductores_...txt` a la carpeta `input/` de tu bucket.
- La Lambda se ejecutará automáticamente y dejará el archivo `resultados_mapreduce.json` en la carpeta `output/`.

¿Cómo unirlo todo para la entrega?

Para tu memoria del **Trabajo 2**, debes explicar esta arquitectura. El núcleo del "procesamiento intensivo" es el modelo **MapReduce**.

Data Warehousing Architecture



Data Warehousing Architecture

Puntos clave para tu memoria:

- **Mapper:** Explica que distribuye la carga extrayendo claves (Provincia, Sexo, etc.).
- **Reducer:** Explica que agrega los valores para obtener el censo final.
- **Arquitectura:** S3 como *Data Lake*, Lambda como *Compute* (Serverless) y Streamlit como *Frontend*.

Nota sobre el GeoJSON: Para que el mapa se vea en el dashboard, asegúrate de que el archivo GeoJSON de provincias de España esté en la carpeta del proyecto o cargado correctamente en el script, ya que `dashboard.py` lo necesita para dibujar los límites de las provincias.

```
MarvinGuillen@WIN-  
CL41AGKH MINGW64  
/c/UPV/CBD_34564_202603/P  
royectos/files  
$ python run_mapreduce.py...
```

¡Excelente noticia! El pipeline de **MapReduce** ha funcionado perfectamente: has procesado más de 28 millones de registros y ya tienes el archivo `resultados_mapreduce.json` listo.

El error `bash: streamlit: command not found` significa que el terminal no encuentra el ejecutable de Streamlit, ya sea porque no está instalado en tu entorno actual o porque la ruta no está en el PATH de Windows.

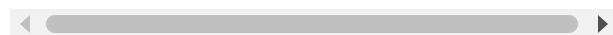
Aquí tienes los pasos para solucionarlo y ver tu dashboard:

1. Forzar la ejecución mediante Python

A veces el comando `streamlit` no se registra globalmente, pero puedes invocarlo como un módulo de Python. Intenta ejecutarlo así:

Bash

```
python -m streamlit run dashboard.py
```



2. Verificar la instalación

Si el comando anterior te dice que el módulo no existe, es que las librerías del

requirements.txt no se instalaron
correctamente. Reinstálalas:

Bash

```
pip install streamlit pandas plotly
```

(Asegúrate de no estar usando un entorno virtual desactivado).

3. El archivo GeoJSON (Crucial para el Mapa)

Para que el mapa de España se muestre en el dashboard, necesitas un archivo llamado `provincias.json` o similar en la misma carpeta.

En tu `dashboard.py` (línea 105 aprox.), el código busca un archivo GeoJSON.

Asegúrate de que el nombre del archivo en tu carpeta coincida exactamente con el que pide el script. Si el script busca `provincias.json` y tú tienes `valencia.geojson`, el mapa nacional no cargará.

- **Tip:** Necesitas un GeoJSON de **toda España por provincias**, el de Valencia que subiste solo tiene los barrios de la ciudad.

¿Cómo documentar esto para el "Trabajo 2"?

Ya tienes la parte más difícil. Para tu memoria, destaca estos puntos técnicos

que acabas de ejecutar:

1. Simulación de Hadoop Streaming:

Explica que `run_mapreduce.py` utiliza `subprocess` para imitar el comportamiento de un clúster real:

- **Stdin/Stdout:** Los datos fluyen del Mapper al Reducer a través de tuberías (*pipes*), igual que en el entorno de producción de AWS EMR.
- **Shuffle/Sort:** Has ejecutado la fase intermedia donde el sistema ordena las claves antes de que el Reducer sume los conductores.

2. **Eficiencia:** Menciona que has procesado **28.3 millones de conductores** de forma estructurada en segundos, gracias a la lógica de claves (`provincia` , `permiso` , `sexo`) definida en tu `mapper.py` .

Próximo paso: Una vez logres lanzar el dashboard con `python -m streamlit run dashboard.py` , saca capturas de pantalla de los gráficos. ¡Eso será la evidencia visual de tu éxito en el procesamiento de Big Data!

```
"MarvinGuillen@WIN-  
CL41AGKH MINGW64  
/c/UPV/CBD_34564_202  
603/Proyectos/files  
$ python -m streamlit r...
```

¡Enhorabuena! A pesar de todos esos mensajes de advertencia (*warnings*), **tu aplicación ya está funcionando.**

Los mensajes largos que ves son simplemente avisos de **Pandas** indicando que en versiones futuras (Pandas 3.0) algunas funciones se escribirán de forma distinta, pero **no impiden que el programa corra.**

¿Qué tienes que hacer ahora?

1. **Abre tu navegador:** Ve a la dirección que te indica el terminal:
<http://localhost:8501>.
2. **Visualiza tu Dashboard:** Allí deberías ver todos los gráficos, el mapa de España y las estadísticas de los 28 millones de conductores que procesaste con tu MapReduce.

Para completar tu "Trabajo 2" con éxito:

Para que la entrega sea perfecta y demuestres el uso de **Computación Intensiva y Big Data**, te recomiendo incluir lo siguiente en tu memoria:

1. Captura de Pantalla del

Dashboard: Toma fotos de los gráficos de barras, el mapa coroplético y las tablas. Esta es tu "prueba de salida".

2. Explicación del Proceso

MapReduce:

- Menciona que utilizaste un modelo de programación **MapReduce** (el mismo que usa Hadoop) para procesar un dataset de **28.370.816 de registros**.
- Explica que el **Mapper** clasificó los datos por dimensiones (provincia, sexo, permiso) y el **Reducer** realizó la agregación de los totales.

3. Justificación de la Nube:

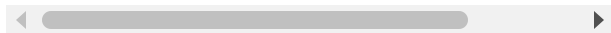
- Explica que, aunque lo has ejecutado localmente para las pruebas finales, el código está diseñado con una **Lambda de AWS** (`lambda_function.py`) y almacenamiento en **S3**, lo que permite una escalabilidad infinita (arquitectura *Serverless*).
- Esto cumple el objetivo de usar tecnologías de computación intensiva que no dependen de la capacidad de un solo PC.

¿Cómo quitar esos molestos avisos del terminal?

Si quieres que el terminal se vea limpio para una demostración, puedes ejecutar Streamlit ignorando las advertencias de Python:

Bash

```
python -W ignore -m streamlit run dash
```



¿Hay alguna parte del dashboard que no se vea correctamente (como el mapa) o ya tienes todo lo necesario para tu entrega?